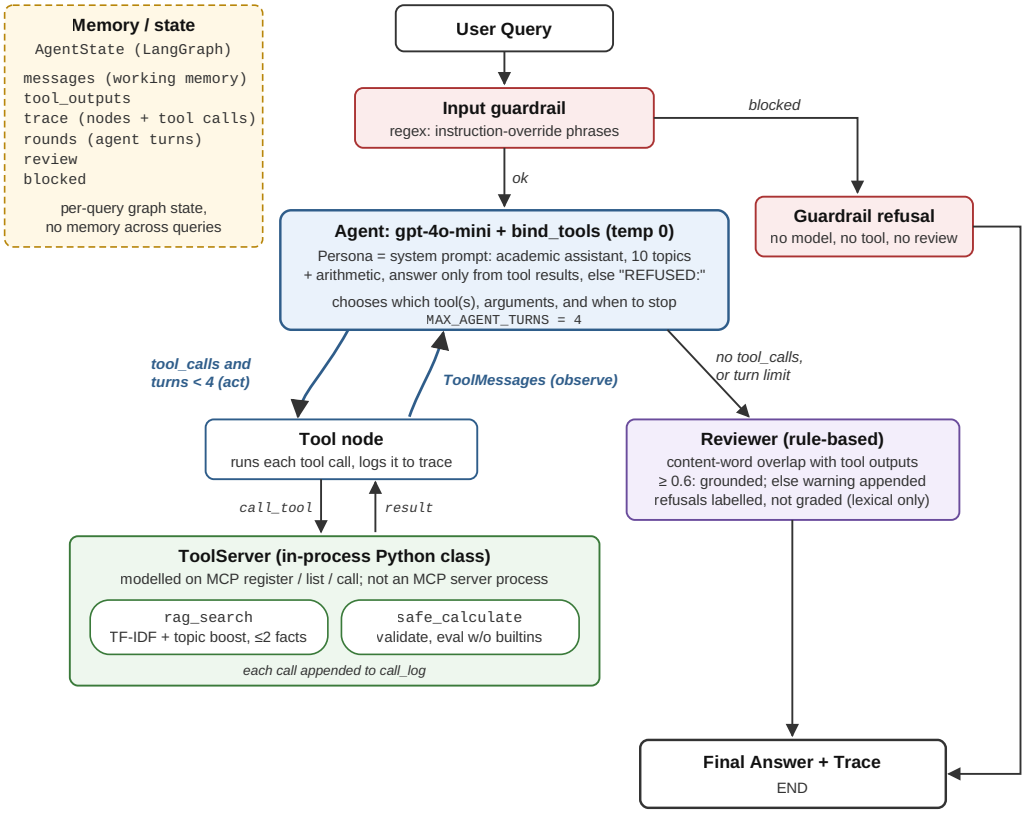
Architecture Diagram: Actual Implementation

Single tool-calling agent in a LangGraph loop, with input guardrail, Reviewer, and in-process tool server
Repository: https://github.com/sandipdey2/agentic-rag-system



Source: https://github.com/sandipdey2/agentic-rag-system

What the code does

- The graph starts at a rule-based input guardrail. A regex match on instruction-override text goes straight to a fixed refusal: no model call, no tool call, no review. There is no Agent Gateway.
- Otherwise the agent (gpt-4o-mini, temperature 0, `bind_tools`) receives the system prompt (the persona) and the query, and decides which tool(s) to call, with what arguments, or whether to answer or refuse (`REFUSED:`).
- Reasoning loop: agent → tool node → agent repeats while the model returns tool calls, with at most `MAX_AGENT_TURNS = 4` agent turns. Tool results return as `ToolMessages`.
- Tools `rag_search` (TF-IDF plus a +0.6 topic boost, up to two facts) and `safe_calculate` (validated, evaluated without built-ins) are LangChain tools whose bodies call `ToolServer.call_tool`. `ToolServer` is an in-process Python class modelled on MCP register / list / call, not an MCP server process.
- The Reviewer computes the share of the answer's content words found in the tool outputs and arguments. At 0.6 or above it marks the answer grounded; below that it appends a warning. Refusals are labelled, not graded. The check is lexical, so it cannot verify reasoning: in the saved run (Python 3.13.9) it marked the wrong two-step Transformer answer ("9 years after 2014", computed as 2023 - 2014) grounded (0.88).
- `AgentState` (messages, tool_outputs, trace, rounds, review, blocked) is per-query graph state. The message list is working memory across loop turns; `run_agent` creates a fresh state for every query, so nothing is remembered across queries. The trace records every node and every tool call with its arguments.